

Soal 1: Analisis Perpindahan Penumpang Antarhalte

Kelompok A12 – TK 1 Analisis Numerik

Rangkuman

Studi ini menentukan distribusi steady state untuk enam matriks perpindahan penumpang kode B, dengan 16 sampai 512 halte. Karena $I - T^T$ singular, satu persamaan diganti dengan syarat skala dan solusi dinormalkan. Kami mengimplementasikan LU dense serta eliminasi pita berpivot parsial untuk matriks dengan lower bandwidth 2 dan upper bandwidth 1. Kedua solver memberi solusi dan residual yang sama pada ketelitian eksperimen. Pada $N = 512$, solver pita memerlukan 60 KiB dan 20.1 ms, dibandingkan 4108 KiB dan 164.2 ms untuk dense. Kasus $N = 256$ hampir terpisah menjadi dua bagian dan menghasilkan kondisi terbesar, $\kappa_1(B) \approx 1.71 \times 10^9$. Halte terakhir memiliki proporsi maksimum pada semua ukuran. Solver pita direkomendasikan untuk koridor besar, sambil tetap memantau kondisi matriks.

Daftar Isi

1	Pendahuluan	2
2	Data dan Validasi	2
3	Formulasi dan Penanganan Singularitas	3
4	Identifikasi Struktur Banded	4
5	Metode Penyelesaian	5
6	Eksperimen Kinerja dan Analisis Kompleksitas	8
7	Analisis Kondisi dan Galat	11
8	Interpretasi Hasil	15
9	Rekomendasi	19
10	Kesimpulan	21
A	Kode Program	22

1 Pendahuluan

Distribusi penumpang jangka panjang pada koridor halte dapat dimodelkan sebagai distribusi stasioner rantai Markov. Tujuan studi ini ialah menghitung distribusi tersebut untuk enam matriks transisi kode B milik kelompok genap A12, dan membandingkan solver LU dense dengan solver berpivot parsial yang memanfaatkan struktur pita. Kami memeriksa validitas data, mengatasi singularitas sistem stasioner dengan satu syarat skala, lalu mengukur waktu, memori, kondisi, dan galat. Hasil dipakai untuk menentukan metode yang tepat ketika jumlah halte membesar.

2 Data dan Validasi

2.1 Deskripsi data

Sebuah koridor memiliki N halte bernomor $1, \dots, N$ sesuai urutan lokasinya. Misalkan X_k adalah halte tempat seorang penumpang berada pada interval waktu ke- k . Matriks transisi $T \in \mathbb{R}^{N \times N}$ berisi $T_{ij} = \Pr(X_{k+1} = j \mid X_k = i)$, yaitu peluang penumpang berpindah dari halte i ke halte j dalam satu interval. Data kode B terdiri atas enam berkas `T_N.csv` untuk $N \in \{16, 32, 64, 128, 256, 512\}$, masing-masing berisi N baris dan N kolom bilangan riil tanpa header. Pada keenam berkas, elemen tak-nol hanya terletak pada kolom $j \in \{i-1, i, i+1, i+2\}$. Dalam satu interval, penumpang hanya dapat mundur satu halte, tetap, atau maju satu hingga dua halte, sehingga setiap baris memuat dua sampai empat elemen tak-nol dan T berstruktur pita (*banded*).

2.2 Prosedur validasi

Setiap matriks dibaca dengan fungsi `baca_T` lalu diperiksa oleh `validasi_T` (Lampiran A). Pemeriksaan pertama menyangkut bentuk data: T harus berukuran $N \times N$ sesuai nama berkas, seluruh elemennya hingga (bukan NaN atau Inf), dan $T_{ij} \geq 0$ karena setiap elemen adalah peluang. Jumlah setiap baris harus satu. Data disimpan sebagai bilangan titik-mengambang, jadi syarat ini diperiksa dengan toleransi, yaitu $\max_i \left| \sum_{j=1}^N T_{ij} - 1 \right| \leq \tau$ dengan $\tau = 10^{-12}$. Nilai τ jauh di atas epsilon mesin presisi ganda, $\varepsilon = 2^{-53} \approx 1.11 \times 10^{-16}$ [3], tetapi cukup kecil untuk mendeteksi kesalahan data.

Syarat berikutnya menjamin distribusi *steady state* tunggal. Rantai Markov berhingga yang *irreducible*, yaitu setiap halte dapat dicapai dari setiap halte lain, memiliki tepat satu distribusi stasioner dan seluruh komponennya positif [1]. Untuk memeriksanya, T dipandang sebagai graf berarah G dengan simpul berupa halte dan sisi $i \rightarrow j$ jika $T_{ij} > 0$. Rantai *irreducible* jika dan hanya jika G terhubung kuat. Hal ini diperiksa dengan dua *breadth-first search* (BFS) [2] dari halte 1. BFS pertama dijalankan pada G untuk memastikan semua halte dapat dicapai dari halte 1, dan BFS kedua pada G dengan arah sisi dibalik untuk memastikan halte 1 dapat dicapai dari semua halte. Sebagai pemeriksaan tambahan, rantai juga harus aperiodik agar distribusi penumpang konvergen ke distribusi stasioner dari kondisi awal apa pun [1]. Untuk rantai *irreducible*, cukup ada satu $T_{ii} > 0$. Karena T disimpan dense, setiap BFS memindai satu baris atau kolom penuh untuk setiap simpul, sehingga biaya validasi $O(N^2)$. Biaya ini termasuk tahap persiapan data, bukan solver.

2.3 Hasil validasi

Tabel 2.1: Hasil validasi matriks transisi. Untuk semua N , dimensi sesuai, seluruh elemen hingga dan nonnegatif, rantai irreducible dan aperiodik, dan setiap baris memuat 2–4 elemen tak-nol.

N	$\max_i \sum_j T_{ij} - 1 $	$\min_i T_{ii}$	$\min\{T_{ij} > 0 : i \neq j\}$	Valid
16	1.11×10^{-16}	0.835	0.0144	Ya
32	2.22×10^{-16}	0.835	0.0147	Ya
64	2.22×10^{-16}	0.835	0.0148	Ya
128	2.22×10^{-16}	0.835	0.0149	Ya
256	2.22×10^{-16}	0.835	1.50×10^{-8}	Ya
512	2.22×10^{-16}	0.835	0.0150	Ya

Tabel 2.1 menunjukkan bahwa keenam matriks lolos seluruh pemeriksaan. Galat jumlah baris maksimum adalah ε untuk $N = 16$ dan $2\varepsilon = 2.22 \times 10^{-16}$ untuk ukuran lainnya, jadi selisih tersebut berasal dari pembulatan dan bukan dari kesalahan data. Seluruh elemen diagonal bernilai paling sedikit 0.835, sehingga setiap rantai aperiodik. Setiap T karena itu memiliki distribusi stasioner tunggal yang positif di semua halte. Sifat irreducible ini juga menjadi dasar bahwa matriks koefisien pada Bagian 3 nonsingular. Fungsi validasi juga diuji pada empat matriks 3×3 , yaitu satu matriks valid dan tiga matriks yang sengaja dibuat salah (ada elemen negatif, satu baris berjumlah 0.9, dan satu halte menyerap sehingga rantai tidak irreducible). Keempat keputusan sesuai harapan (`hasil/uji_validasi.csv`), jadi fungsi tersebut tidak sekadar meloloskan semua data.

Kolom keempat Tabel 2.1 memperlihatkan satu pengecualian. Pada $N = 256$, peluang pindah antara halte 128 dan 129 hanya berorde 10^{-8} , sedangkan peluang pindah positif terkecil pada matriks lain sekitar 0.014. Total peluang lintas dari halte 1–128 ke halte 129–256, yaitu $\sum_{i \leq 128 < j} T_{ij}$, hanya sekitar 9.0×10^{-8} , dan arah sebaliknya juga sekitar 9.0×10^{-8} . Matriks T_{256} tetap valid karena peluang itu positif dan rantai tetap irreducible. Namun, koridornya hampir terpecah menjadi dua blok. Dampaknya terhadap kondisi sistem linear dibahas pada Bagian 7.

3 Formulasi dan Penanganan Singularitas

Misalkan $\pi^{(k)} \in \mathbb{R}^N$ adalah vektor kolom dengan $\pi_i^{(k)} = \Pr(X_k = i)$. Hukum peluang total memberi $\pi_j^{(k+1)} = \sum_i \pi_i^{(k)} T_{ij}$, atau $\pi^{(k+1)} = T^\top \pi^{(k)}$. Distribusi steady state π adalah distribusi yang tidak berubah oleh satu langkah transisi, sehingga memenuhi

$$T^\top \pi = \pi, \quad \mathbf{1}^\top \pi = 1, \quad \pi_i \geq 0, \quad (3.1)$$

dengan $\mathbf{1} \in \mathbb{R}^N$ vektor yang seluruh elemennya satu. Dengan mendefinisikan $A = I - T^\top$, persamaan pertama menjadi $A\pi = 0$.

Matriks A singular. Setiap baris T berjumlah satu, yaitu $T\mathbf{1} = \mathbf{1}$, sehingga

$$\mathbf{1}^\top A = \mathbf{1}^\top - (T\mathbf{1})^\top = \mathbf{0}^\top.$$

Jumlah seluruh baris A adalah vektor nol. Baris-baris A bergantung linear dan $\det A = 0$, jadi eliminasi Gauss atau faktorisasi LU tidak dapat langsung dipakai pada A . Hubungan

yang sama menunjukkan bahwa baris pertama A sama dengan $-\sum_{i \geq 2} A_{i,:}$. Persamaan pertama dari $A\pi = 0$ tidak membawa informasi baru [4], dan dapat diganti dengan syarat skala $z_1 = 1$:

$$B_{1,:} = e_1^\top, \quad B_{i,:} = A_{i,:} \ (i = 2, \dots, N), \quad b = e_1, \quad Bz = b, \quad (3.2)$$

dengan $e_1 = (1, 0, \dots, 0)^\top$.

Matriks B nonsingular. Ekspansi kofaktor sepanjang baris pertama memberi $\det B = \det A_{2:N, 2:N}$, yaitu determinan A setelah baris dan kolom halte 1 dibuang. Submatriks ini sama dengan $I - \tilde{T}^\top$, dengan $\tilde{T}$ matriks transisi tanpa halte 1. Karena rantai irreducible, penumpang di halte mana pun pada akhirnya mencapai halte 1. Peluang untuk terus berada di halte $2, \dots, N$ menuju nol, yaitu $\tilde{T}^k \rightarrow 0$. Akibatnya 1 bukan nilai eigen $\tilde{T}$, sehingga $I - \tilde{T}^\top$ dan B nonsingular [4].

Solusi z masih perlu dinormalkan. Distribusi tunggal π memenuhi $A_{i,:}\pi = 0$ untuk $i \geq 2$ dan $\pi_1 > 0$, jadi solusi tunggal $Bz = b$ adalah $z = \pi/\pi_1$. Baris pertama hanya memilih skala $z_1 = 1$, sehingga umumnya $\mathbf{1}^\top z \neq 1$. Normalisasi

$$\pi = \frac{z}{\mathbf{1}^\top z} \quad (3.3)$$

mengembalikan syarat $\mathbf{1}^\top \pi = 1$. Syarat $\pi_i \geq 0$ ikut terpenuhi karena $z = \pi/\pi_1 > 0$.

4 Identifikasi Struktur Banded

Untuk matriks $M \in \mathbb{R}^{N \times N}$, *lower bandwidth* p dan *upper bandwidth* q adalah

$$p = \max\{i - j : M_{ij} \neq 0\}, \quad q = \max\{j - i : M_{ij} \neq 0\}, \quad (4.1)$$

dengan nilai 0 jika himpunannya kosong. Matriks pita hanya memiliki elemen tak-nol pada p diagonal di bawah dan q diagonal di atas diagonal utama. Fungsi `deteksi_bandwidth` menghitung (4.1) secara otomatis dari posisi elemen tak-nol. Untuk keenam ukuran diperoleh $p = 2$ dan $q = 1$, baik untuk A maupun B . Jadi B *bukan* tridiagonal.

Struktur ini dapat diturunkan langsung. Untuk $i \geq 2$, $B_{ij} = \delta_{ij} - T_{ji}$ dengan δ_{ij} delta Kronecker, sehingga elemen tak-nol T pada posisi $j - i = d$ muncul di B pada posisi $j - i = -d$. Karena posisi tak-nol T adalah $d \in \{-1, 0, 1, 2\}$, baris $i \geq 2$ dari B hanya memuat

$$B_{i,i-2} = -T_{i-2,i}, \quad B_{i,i-1} = -T_{i-1,i}, \quad B_{ii} = 1 - T_{ii}, \quad B_{i,i+1} = -T_{i+1,i}. \quad (4.2)$$

Penggantian baris pertama dengan $e_1^\top$ tetap mempertahankan struktur pita. Vektor $e_1^\top$ hanya tak-nol pada diagonal, jadi penggantian itu dapat menghapus elemen tak-nol tetapi tidak pernah menambah elemen di luar pita.

Matriks dense membutuhkan N^2 bilangan, sedangkan pita B hanya $(p + q + 1)N$. Solver dengan *partial pivoting* membutuhkan sedikit ruang tambahan. Pertukaran baris k dengan baris $r \leq k + p$ membawa elemen tak-nol sampai kolom $r + q \leq k + p + q$, sehingga upper bandwidth faktor U dapat naik dari q menjadi $p + q$ (*fill-in*), sementara lower bandwidth L tetap p [5]. Kami menyimpan B dalam array $AB \in \mathbb{R}^{(2p+q+1) \times N}$ dengan format penyimpanan pita LAPACK [6]: B_{ij} berada di $AB_{p+q+1+i-j, j}$, sehingga setiap diagonal B menjadi satu baris AB dan p baris teratas dicadangkan untuk fill-in. Untuk $p = 2$ dan $q = 1$, array ini berukuran $6 \times N$.

Tabel 4.1: Kebutuhan memori penyimpanan B dalam byte (`float64`, 8 byte per elemen): dense N^2 , pita $(p + q + 1)N$, dan pita dengan ruang fill-in $(2p + q + 1)N$. Rasio = dense dibagi pita dengan fill-in.

N	Dense	Pita	Pita + fill-in	Rasio
16	2 048	512	768	2.7
32	8 192	1 024	1 536	5.3
64	32 768	2 048	3 072	10.7
128	131 072	4 096	6 144	21.3
256	524 288	8 192	12 288	42.7
512	2 097 152	16 384	24 576	85.3

Tabel 4.1 menunjukkan bahwa rasio penyimpanan dense terhadap pita dengan ruang fill-in sama dengan $N/6$. Keunggulan penyimpanan pita tumbuh linear terhadap N . Pada $N = 512$, matriks dense membutuhkan 2 MiB, sedangkan pita hanya 24 KiB, atau sekitar 85 kali lebih kecil.

5 Metode Penyelesaian

Kedua solver diimplementasikan sendiri dalam `soal1/soal1.ipynb`. NumPy hanya dipakai untuk array, *indexing*, dan aritmetika dasar; tidak ada pustaka penyelesaian SPL atau faktorisasi di dalam solver. Fungsi `np.linalg.solve` hanya dipakai pada sel verifikasi yang diberi label terpisah. Matriks permutasi P disimpan sebagai vektor indeks.

5.1 LU dense dengan partial pivoting

Solver (a) menghitung $PB = LU$ dengan L segitiga bawah berdiagonal satu, U segitiga atas, dan P matriks permutasi [3]. Pada langkah k , baris dengan $|B_{ik}|$ terbesar ($i \geq k$) ditukar ke posisi k . Pertukaran dilakukan pada seluruh baris, termasuk multiplier yang sudah tersimpan, agar L konsisten dengan P akhir. Multiplier $\ell_{ik} = B_{ik}/B_{kk}$ disimpan di tempat elemen yang dieliminasi, sehingga L dan U berbagi satu array $N \times N$. Sistem kemudian diselesaikan dengan substitusi maju $Ly = Pb$ dan substitusi mundur $Uz = y$. Langkah-langkah ini dirangkum pada Kode 5.1 dan diimplementasikan pada fungsi `lu_dense` dan `solve_lu`.

Kode 5.1 LU dense dengan *partial pivoting* dan penyelesaian $Bz = b$

Masukan: $B \in \mathbb{R}^{N \times N}$ nonsingular, $b \in \mathbb{R}^N$

Keluaran: $z \in \mathbb{R}^N$ dengan $Bz = b$

```
1:  $M \leftarrow B$  (salinan);  $perm \leftarrow (1, 2, \dots, N)$ 
2: for  $k = 1$  to  $N - 1$  do
3:    $r \leftarrow \arg \max_{k \leq i \leq N} |M_{ik}|$ ; if  $M_{rk} = 0$  then error “singular”
4:   Tukar baris  $M_{k,:} \leftrightarrow M_{r,:}$  dan  $perm_k \leftrightarrow perm_r$  ▷ seluruh baris
5:   for  $i = k + 1$  to  $N$  do
6:      $M_{ik} \leftarrow M_{ik}/M_{kk}$  ▷ multiplier  $\ell_{ik}$ 
7:      $M_{i,k+1:N} \leftarrow M_{i,k+1:N} - M_{ik} M_{k,k+1:N}$ 
8:   end for
9: end for
10:  $y \leftarrow b_{perm}$  ▷  $y = Pb$ 
11: for  $i = 2$  to  $N$  do ▷ substitusi maju  $Ly = Pb$ 
12:    $y_i \leftarrow y_i - \sum_{j < i} M_{ij} y_j$ 
13: end for
14: for  $i = N$  downto  $1$  do ▷ substitusi mundur  $Uz = y$ 
15:    $z_i \leftarrow (y_i - \sum_{j > i} M_{ij} z_j)/M_{ii}$ 
16: end for
17: return  $z$ 
```

5.2 Solver banded bergaya Thomas

Solver (b) harus berjalan dalam $O(N)$ waktu dan memori. Algoritma Thomas klasik memenuhi syarat ini, tetapi hanya berlaku untuk matriks tridiagonal ($p = q = 1$), sedangkan B memiliki $p = 2$ dan $q = 1$. Solver (b) karena itu berupa perluasannya untuk matriks pita umum, yaitu eliminasi Gauss dengan partial pivoting yang hanya bekerja di dalam pita [5], langsung pada array AB dari Bagian 4. Algoritmanya ada pada Kode 5.2 (fungsi `bentuk_B_band`, `lu_band`, dan `solve_band`).

Ada tiga hal yang membuat solver ini berjalan dalam $O(N)$. Pertama, pita diambil langsung dari T : fungsi `bentuk_B_band` mengisi AB menurut (4.2) dan hanya mengakses $N(p + q + 1)$ elemen T , jadi matriks B dense, L , U , maupun P berukuran $N \times N$ tidak pernah dibentuk. Kedua, pivot hanya dicari di baris $k, \dots, \min(N, k + p)$ karena di bawahnya kolom k sudah nol, sedangkan pertukaran dan pembaruan hanya menyentuh kolom $k, \dots, \min(N, k + p + q)$. Ketiga, multiplier langkah sebelumnya tidak ikut ditukar seperti pada Kode 5.1, karena tersimpan di kolom lain AB . Indeks pertukaran setiap langkah dicatat di vektor *piv*, lalu diulang dengan urutan yang sama pada ruas kanan saat substitusi maju. Hasilnya setara dengan $PB = LU$.

Kode 5.2 Faktorisasi dan penyelesaian banded dengan *partial pivoting*. Notasi B_{ij} berarti elemen yang disimpan di $AB_{p+q+1+i-j, j}$.

Masukan: $AB \in \mathbb{R}^{(2p+q+1) \times N}$ berisi pita B , $p, q, b \in \mathbb{R}^N$

Keluaran: $z \in \mathbb{R}^N$ dengan $Bz = b$

```

1: for  $k = 1$  to  $N$  do
2:    $i_{\max} \leftarrow \min(N, k + p); \quad j_{\max} \leftarrow \min(N, k + p + q)$ 
3:    $r \leftarrow \arg \max_{k \leq i \leq i_{\max}} |B_{ik}|$ ; if  $B_{rk} = 0$  then error “singular”
4:    $piv_k \leftarrow r$ ; tukar  $B_{kj} \leftrightarrow B_{rj}$  untuk  $j = k, \dots, j_{\max}$ 
5:   for  $i = k + 1$  to  $i_{\max}$  do
6:      $B_{ik} \leftarrow B_{ik}/B_{kk}$  ▷ multiplier  $\ell_{ik}$ 
7:      $B_{ij} \leftarrow B_{ij} - B_{ik}B_{kj}$  untuk  $j = k + 1, \dots, j_{\max}$ 
8:   end for
9: end for
10:  $y \leftarrow b$ 
11: for  $k = 1$  to  $N$  do ▷ maju: ulangi pertukaran dan eliminasi
12:   tukar  $y_k \leftrightarrow y_{piv_k}$ ;  $y_i \leftarrow y_i - B_{ik}y_k$  untuk  $i = k + 1, \dots, \min(N, k + p)$ 
13: end for
14: for  $i = N$  downto  $1$  do ▷ mundur:  $U$  selebar  $p + q$ 
15:    $z_i \leftarrow (y_i - \sum_{j=i+1}^{\min(N, i+p+q)} B_{ij}z_j)/B_{ii}$ 
16: end for
17: return  $z$ 

```

5.3 Uji kebenaran

Sebelum dipakai pada data, kedua solver diuji pada matriks kecil yang hasilnya dapat dicek manual (Tabel 5.1). Pada uji 2, hitungan tangan dengan dua kali pertukaran baris memberi

$$L = \begin{pmatrix} 1 & 0 & 0 \\ 0.25 & 1 & 0 \\ 0.5 & 2/3 & 1 \end{pmatrix}, \quad U = \begin{pmatrix} 8 & 7 & 9 \\ 0 & -0.75 & -1.25 \\ 0 & 0 & -2/3 \end{pmatrix}, \quad perm = (3, 1, 2),$$

dengan $perm$ urutan baris B setelah permutasi P . Kedua solver menghasilkan faktor yang sama persis.

Tabel 5.1: Uji kebenaran pada matriks kecil.

Kasus	Solver	Hasil
1. Matriks valid dan tiga matriks rusak 3×3 (Bagian 2)	validasi	keempat keputusan sesuai
2. $B = \begin{pmatrix} 2 & 1 & 1 \\ 4 & 3 & 3 \\ 8 & 7 & 9 \end{pmatrix}$, $b = (3, 7, 19)^\top$	dense, banded ($p = q = 2$)	L, U, P sama dengan hitungan tangan; $\max PB - LU = 0$; $z = (1, -1, 2)^\top$
3. Matriks 4×4 dengan $B_{11} = 0$	dense	baris 1 dan 2 ditukar; galat solusi 4.4×10^{-16}
4. Matriks pita 8×8 , $p = 2$, $q = 1$, diagonal diperkecil 100 kali	banded vs dense	7 pertukaran baris, 8 elemen fill-in, U melebar tepat sampai $p + q = 3$; P dan U identik dengan solver dense; galat solusi 5.3×10^{-15}
5. Matriks singular $\begin{pmatrix} 1 & 2 \\ 2 & 4 \end{pmatrix}$	dense, banded	keduanya menolak karena pivot nol
6. Rantai <i>birth-death</i> 3×3 dengan $\pi = (\frac{1}{4}, \frac{1}{2}, \frac{1}{4})^\top$	dense, banded	π tepat sama

Pada keenam data, pivoting tidak pernah menukar baris (`hasil/akurasi_dense.csv` dan `hasil/akurasi_band.csv`). Penyebabnya, B dominan diagonal per kolom. Pada kolom $k \geq 2$ berlaku $|B_{kk}| = 1 - T_{kk} = \sum_{i \neq k} T_{ki} \geq \sum_{i \neq k, i \geq 2} |B_{ik}|$, dan pada kolom pertama $B_{11} = 1 \geq T_{12} + T_{13}$. Eliminasi Gauss mempertahankan sifat ini, sehingga partial pivoting selalu memilih elemen diagonal [5]. Jalur pertukaran baris karena itu hanya teruji lewat uji 2–4 pada Tabel 5.1.

Tanpa pertukaran baris, kedua solver melakukan operasi titik-mengambang yang sama pada angka yang sama; solver dense hanya menambah operasi terhadap elemen nol. Hasilnya, π dari solver banded identik dengan π dari solver dense, dengan selisih maksimum 0 pada mesin kami, karena tanpa pertukaran baris kedua solver melakukan operasi yang sama. Sebagai pembandingan independen, selisih terhadap `np.linalg.solve` paling besar 2.6×10^{-16} untuk $N \neq 256$ dan 6.6×10^{-13} untuk $N = 256$ (`hasil/verifikasi*_numpy.csv`). Selisih yang lebih besar pada $N = 256$ sejalan dengan struktur yang hampir terpecah pada Subbagian 2.3 dan dianalisis pada Bagian 7.

6 Eksperimen Kinerja dan Analisis Kompleksitas

6.1 Rancangan eksperimen

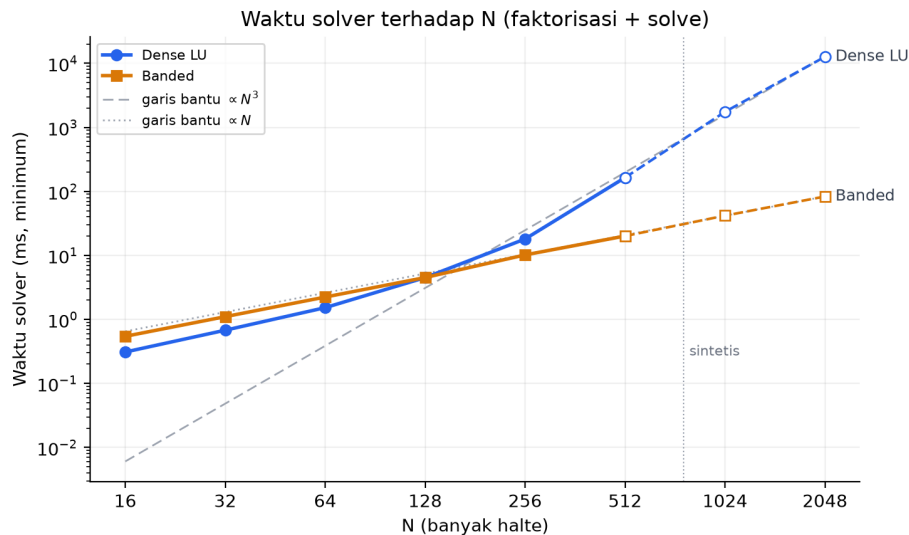
Eksperimen dijalankan pada AMD Ryzen 7 7435HS dengan RAM 16 GB, Windows 11, Python 3.14.4, dan NumPy 2.5.3. Waktu solver adalah waktu faktorisasi, substitusi, dan normalisasi (3.3). Waktu membaca CSV dan persiapan diukur terpisah. Persiapan solver dense adalah pembentukan B dense, yang berbiaya $O(N^2)$. Persiapan solver banded adalah deteksi p dan q dari posisi tak-nol T , yang juga $O(N^2)$ karena data masukan dense, ditambah ekstraksi pita yang $O(N)$. Waktu diukur dengan `time.perf_counter`. Setelah satu kali pemanasan, setiap pengukuran diulang 15 kali dan diambil nilai minimumnya, karena gangguan dari sistem operasi hanya dapat menambah waktu. Memori diukur dengan dua cara: puncak alokasi selama solver berjalan menurut `tracemalloc`, dan total

.nbytes array yang disimpan solver (koefisien, ruas kanan, faktor, vektor pivot, dan solusi). Seluruh hasil tersimpan di `hasil/benchmark.csv`.

6.2 Hasil

Tabel 6.1: Waktu (ms, minimum dari 15 ulangan) dan memori (KiB) untuk data soal.

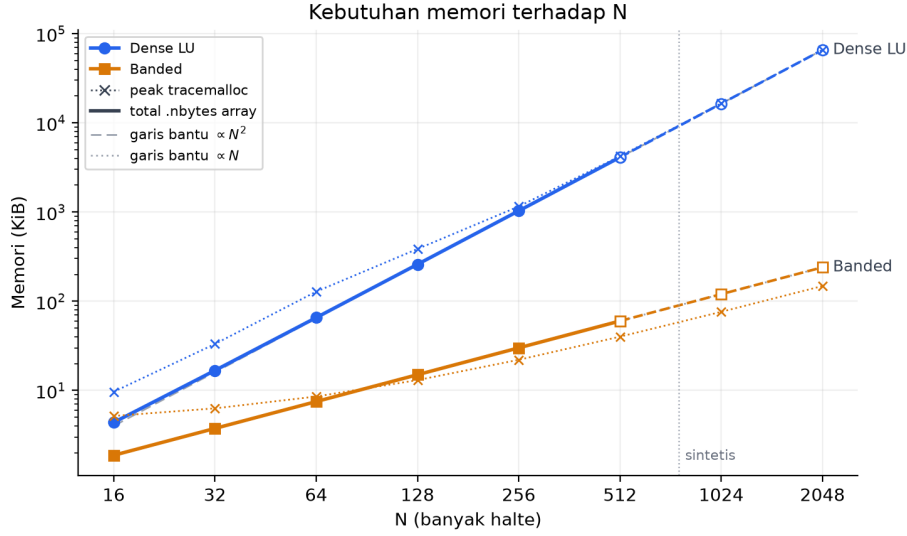
N	Baca CSV	Dense (ms)		Banded (ms)		.nbytes		Peak	
		persiapan	solver	persiapan	solver	dense	banded	dense	banded
16	0.40	0.01	0.31	0.04	0.54	4.4	1.9	9.7	5.2
32	0.47	0.01	0.68	0.08	1.10	16.8	3.8	33.2	6.3
64	0.81	0.01	1.53	0.16	2.23	65.5	7.5	128.2	8.5
128	2.05	0.05	4.50	0.37	4.51	259.0	15.0	385.1	13.0
256	7.35	0.29	18.04	0.96	10.18	1030.0	30.0	1153.6	22.0
512	29.40	3.19	164.22	2.93	20.09	4108.0	60.0	4225.9	40.0



Gambar 6.1: Waktu solver terhadap N (skala log-log). Marker kosong dan garis putus-putus adalah rantai sintetis $N = 1024$ dan 2048 ; garis abu-abu adalah garis bantu $\propto N^3$ dan $\propto N$.

Tabel 6.1 dan Gambar 6.1 menunjukkan bahwa keunggulan solver banded baru muncul pada N sedang. Untuk $N \leq 64$, solver banded 1.5–1.8 kali lebih lambat daripada solver dense, dan pada $N = 128$ keduanya setara. Setiap langkah solver banded berupa perulangan Python kecil dengan biaya tetap, sedangkan setiap langkah solver dense adalah satu operasi vektor NumPy yang dieksekusi dalam kode terkompilasi. Pada N kecil, waktu lebih ditentukan oleh banyaknya langkah daripada banyaknya flop. Mulai $N = 256$ solver banded lebih cepat, yaitu 1.9 kali pada $N = 256$ dan 8.2 kali pada $N = 512$. Pada $N = 512$, waktu membaca CSV (29.4 ms) bahkan sudah melebihi waktu solver banded (20.1 ms). Untuk melihat tren pada N yang lebih besar, kedua solver juga dijalankan pada rantai sintetis $N = 1024$ dan 2048 dengan pola pita yang sama (bukan data soal,

lima ulangan). Pada $N = 2048$, solver dense membutuhkan 12.7 s, sedangkan solver banded hanya 83 ms.



Gambar 6.2: Memori terhadap N . Garis tebal menunjukkan total `.nbytes` array yang disimpan solver; garis bertitik dengan marker silang menunjukkan puncak `tracemalloc`.

Perbedaan memori pada Gambar 6.2 lebih tegas daripada perbedaan waktu. Total `.nbytes` solver dense naik empat kali setiap N digandakan, sedangkan solver banded hanya naik dua kali. Pada $N = 512$, solver dense menyimpan 4108 KiB dan solver banded 60 KiB, sekitar 68 kali lebih kecil. Puncak `tracemalloc` mengikuti pola yang sama.

6.3 Analisis kompleksitas

Banyaknya operasi dihitung dalam flop: satu flop adalah satu pasangan perkalian dan pengurangan, atau satu pembagian. Pada solver dense, langkah k mengeliminasi $N - k$ baris dan tiap baris memperbarui $N - k$ elemen, sehingga faktorisasi memerlukan $\sum_{k=1}^{N-1} (N - k)^2 \approx N^3/3$ flop [3]. Substitusi maju dan mundur masing-masing sekitar $N^2/2$ flop. Array faktornya berukuran N^2 . Jadi solver dense membutuhkan $O(N^3)$ waktu dan $O(N^2)$ memori.

Pada solver banded, setiap kolom hanya mengeliminasi p baris, dan setiap baris hanya memperbarui $p + q$ elemen karena faktor U melebar akibat pivoting. Faktorisasi memerlukan sekitar

$$N p (p + q) \quad (6.1)$$

flop, yaitu $6N$ untuk $p = 2$ dan $q = 1$; taksiran $(p + q)^2 N/4$ yang lebih kecil berlaku untuk eliminasi pita tanpa pivoting. Substitusi maju dan mundur hanya menyentuh elemen di dalam pita, sehingga biayanya $O(N(p + q))$. Memori yang dibutuhkan adalah array AB berukuran $(2p + q + 1)N$. Untuk p dan q tetap, waktu dan memori sama-sama $O(N)$, sesuai yang dituntut dari solver (b). Sebagai gambaran, pada $N = 512$ faktorisasi dense memerlukan sekitar 4.5×10^7 flop, sedangkan faktorisasi banded hanya 3072 flop (`hasil/flop_teorii.csv`).

Tabel 6.2: Kemiringan garis log-log (pangkat empiris N) terhadap nilai teoretis. Kolom terakhir memakai $N = 256, 512$ dan dua titik sintetis.

Besaran	Teori	$N = 16\text{--}512$	$N = 128\text{--}512$	$N = 256\text{--}2048$
Waktu solver dense	3	1.74	2.59	3.18
Waktu solver banded	1	1.05	1.08	1.02
Flop faktorisasi dense	3	3.00	3.00	3.00
Flop faktorisasi banded	1	1.00	1.00	1.00
<code>.nbytes</code> dense	2	1.98	1.99	2.00
<code>.nbytes</code> banded	1	1.00	1.00	1.00
Peak dense	2	1.74	1.73	1.95
Peak banded	1	0.59	0.81	0.92

Tabel 6.2 membandingkan teori dengan eksperimen melalui kemiringan garis terbaik pada skala log-log, yang dihitung dengan rumus regresi linear sederhana. Kemiringan waktu solver banded sekitar 1 pada semua rentang, sesuai $O(N)$. Kemiringan waktu solver dense hanya 1.71 pada seluruh data karena pada N kecil biaya tetap setiap langkah masih dominan, lalu naik menjadi 2.56 untuk $N \geq 128$ dan 3.18 untuk $N \geq 256$, mendekati N^3 . Kemiringan flop faktorisasi dan `.nbytes` sama dengan teori: 3 dan 2 untuk dense, 1 untuk banded. Eksperimen ini mengonfirmasi analisis di atas: solver banded linear dalam waktu dan memori, sedangkan solver dense kuadratik dalam memori dan mendekati kubik dalam waktu.

7 Analisis Kondisi dan Galat

7.1 Ukuran yang dihitung

Bagian ini menimbang tiga hal: kondisi B sebagai sifat masalah, galat residual sebagai sifat algoritma, dan galat maju sebagai akibat dari keduanya. Kondisi B diukur dengan bilangan kondisi norma-1

$$\kappa_1(B) = \|B\|_1 \|B^{-1}\|_1, \quad \|M\|_1 = \max_j \sum_i |M_{ij}|, \quad (7.1)$$

yang dipakai untuk seluruh N agar nilainya dapat dibandingkan antarukuran. Norma-1 dipilih karena nilainya diperoleh eksak dari jumlah mutlak kolom, sedangkan norma-2 memerlukan nilai singular yang tidak dapat dihitung tanpa pustaka faktorisasi [3]. Matriks B^{-1} dibentuk dengan menyelesaikan $BX = I$ memakai faktor $PB = LU$ dari solver dense, yaitu substitusi maju dan mundur untuk semua kolom sekaligus (Kode 7.1; fungsi `invers_dari_LU` dan `bilangan_kondisi_1`). Biayanya $O(N^3)$, yang masih dapat diterima untuk $N \leq 2048$. Untuk N yang jauh lebih besar, κ_1 cukup ditaksir dengan estimator Hager–Higham, yang hanya memerlukan penyelesaian sistem dengan B dan B^T memakai faktor yang sudah ada [7]. Fungsi ini diuji pada matriks 3×3 dari Tabel 5.1 (uji 2): `invers` sama dengan hitungan tangan dan $\kappa_1 = 14 \times 5.5 = 77$.

Kode 7.1 Bilangan kondisi $\kappa_1(B)$ dari faktor $PB = LU$

Masukan: $B \in \mathbb{R}^{N \times N}$, faktor L, U dan vektor permutasi $perm$ dari Kode 5.1

Keluaran: $\kappa_1(B)$

```

1:  $Y \leftarrow I_{perm,:}$   $\triangleright Y = PI$ 
2: for  $i = 2$  to  $N$  do  $\triangleright$  substitusi maju untuk semua kolom:  $LY = PI$ 
3:    $Y_{i,:} \leftarrow Y_{i,:} - \sum_{j < i} L_{ij} Y_{j,:}$ 
4: end for
5: for  $i = N$  downto  $1$  do  $\triangleright$  substitusi mundur:  $UX = Y$ , sehingga  $X = B^{-1}$ 
6:    $X_{i,:} \leftarrow (Y_{i,:} - \sum_{j > i} U_{ij} X_{j,:}) / U_{ii}$ 
7: end for
8: return  $(\max_j \sum_i |B_{ij}|) (\max_j \sum_i |X_{ij}|)$ 

```

Selain $r = \|T^\top \hat{\pi} - \hat{\pi}\|_2$ dan $e_{\text{norm}} = |\mathbf{1}^\top \hat{\pi} - 1|$ yang diminta soal, kami menghitung dua ukuran tambahan. Yang pertama adalah galat mundur relatif

$$\eta = \frac{\|b - B\hat{z}\|_1}{\|B\|_1 \|\hat{z}\|_1}, \quad (7.2)$$

yaitu seberapa kecil gangguan relatif pada B yang membuat $\hat{z}$ menjadi solusi eksak [7]. Yang kedua adalah galat maju $\delta = \|\hat{\pi} - \pi^*\|_1$ terhadap solusi acuan π^* . Karena $\|\pi^*\|_1 = 1$, δ adalah galat relatif norma-1. Acuan harus lebih akurat daripada presisi ganda. Membandingkan kedua solver satu sama lain tidak membantu karena hasilnya identik (Subbagian 7.4), dan `np.linalg.solve` juga berasal dari eliminasi dengan pembulatan. Karena itu π^* dihitung dengan perbaikan iteratif dengan residual eksak (Kode 7.2). $\hat{z}$ diubah menjadi bilangan rasional, residual $b - B\hat{z}$ dihitung eksak dengan modul `fractions` Python (hanya di dalam pita), koreksi dihitung dengan faktor LU yang sama, dan langkah ini diulang empat kali. Setiap iterasi memperkecil galat kira-kira dengan faktor $\kappa_1 \varepsilon$ [7]. Pada kasus terburuk ($\kappa_1 \varepsilon \approx 1.9 \times 10^{-7}$) galat acuan setelah empat iterasi karena itu jauh di bawah δ yang diukur.

Kode 7.2 Solusi acuan π^* dan galat maju δ dengan perbaikan iteratif dan residual eksak

Masukan: B (pita p, q), b , faktor LU dan $perm$, solusi $\hat{z}$ dan $\hat{\pi}$, jumlah iterasi $K = 4$

Keluaran: $\delta = \|\hat{\pi} - \pi^*\|_1$

```

1:  $\zeta \leftarrow \hat{z}$  sebagai bilangan rasional (tepat)
2: for  $k = 1$  to  $K$  do
3:    $\rho \leftarrow b - B\zeta$   $\triangleright$  aritmetika rasional, hanya elemen di dalam pita
4:    $d \leftarrow$  solusi  $LUd = P\rho$  dengan  $\rho$  dibulatkan ke presisi ganda  $\triangleright$  substitusi Kode 5.1
5:    $\zeta \leftarrow \zeta + d$   $\triangleright$  penjumlahan eksak
6: end for
7:  $\pi^* \leftarrow \zeta / \mathbf{1}^\top \zeta$   $\triangleright$  eksak
8: return  $\sum_i |\hat{\pi}_i - \pi_i^*|$   $\triangleright$  dihitung eksak, lalu dibulatkan

```

Ketiga ukuran dihubungkan oleh pertidaksamaan sederhana. Karena $\hat{z} - z = B^{-1}(B\hat{z} - b)$,

$$\frac{\|\hat{z} - z\|_1}{\|\hat{z}\|_1} \leq \|B^{-1}\|_1 \frac{\|b - B\hat{z}\|_1}{\|\hat{z}\|_1} = \kappa_1(B) \eta. \quad (7.3)$$

Galat maju dengan demikian dibatasi oleh hasil kali sifat masalah (κ_1) dan sifat algoritma (η). Eliminasi Gauss yang stabil mundur memberi η sekitar $\varepsilon = 2^{-53}$ [7], sehingga $\kappa_1 \varepsilon$

lazim dipakai sebagai tolok ukur galat maju. Kami melaporkan keduanya. Residual pada (7.2) dihitung dalam presisi ganda, jadi nilainya hanya dapat dibaca sampai orde ε dan (7.3) dipakai sebagai petunjuk orde, bukan batas yang pasti.

7.2 Hubungan $\kappa_1(B)$ dengan waktu pencapaian halte 1

Untuk matriks B ini, $\kappa_1(B)$ memiliki tafsiran fisik yang eksak. Misalkan m_j adalah rata-rata banyaknya interval waktu yang diperlukan penumpang dari halte j untuk pertama kali tiba di halte 1 (*hitting time*).

Proposisi. $\kappa_1(B) = \|B\|_1 \max\{1/\pi_1, \max_{j \geq 2} m_j\}$.

Bukti. Semua elemen B^{-1} nonnegatif (ditunjukkan di bawah), sehingga $\|B^{-1}\|_1$ adalah jumlah kolom terbesar. Kolom pertama adalah $B^{-1}e_1 = z = \pi/\pi_1$ (Bagian 3), yang berjumlah $1/\pi_1$. Untuk $j \geq 2$, kolom $x = B^{-1}e_j$ memenuhi $x_1 = 0$ (baris pertama B), dan komponen sisanya memenuhi $(I - \tilde{T}^\top)x_{2:N} = e_j$ dengan $\tilde{T}$ seperti pada Bagian 3. Karena $\tilde{T}^k \rightarrow 0$, $x_{2:N} = \sum_{k \geq 0} (\tilde{T}^\top)^k e_j \geq 0$. Komponen ke- i adalah rata-rata banyaknya kunjungan ke halte i sebelum penumpang yang berangkat dari halte j tiba di halte 1, sehingga jumlah komponennya adalah m_j [1]. $\square$

Proposisi ini menyatakan bahwa buruknya kondisi B sama dengan lamanya penumpang dari halte terjauh mencapai halte 1. Dua akibatnya menjelaskan Tabel 7.1.

Pertama, rantai pada koridor ini berdifusi. Peluang pindah di bagian dalam koridor adalah sekitar 0.09 (mundur satu halte), 0.06 (maju satu), dan 0.015 (maju dua), sehingga variansi perpindahan per interval adalah $\sigma^2 = 0.09 + 0.06 + 4(0.015) \approx 0.21$ dengan kecenderungan arah (*drift*) hampir nol (Subbagian 8.3). Untuk langkah acak seperti ini, waktu pencapaian dari ujung jauh koridor berorde N^2/σ^2 [1], sehingga $\kappa_1(B) = O(N^2)$. Hal ini sesuai dengan Tabel 7.1: κ_1 naik 3.9–4.0 kali setiap N digandakan dari 16 sampai 128, dan κ_1/N^2 hampir konstan (6.2–6.6 pada data). Pada rantai sintetis dengan drift nol, $\kappa_1/N^2 = 5.11$ –5.12, sama dengan $\|B\|_1/\sigma^2 = 1.075/0.21$, dengan $\|B\|_1 = 1.075$ dari kolom pertama ($1 + 0.06 + 0.015$). Nilai pada data sedikit lebih besar. Penyebab yang mungkin adalah peluang mundur di dekat halte 1 pada data yang lebih kecil daripada 0.09 (misalnya $T_{21} = 0.0651$), sehingga penumpang lebih lama mencapai halte 1.

Kedua, pada $N = 256$ penumpang di halte 129–256 hanya dapat mencapai halte 1 dengan melintasi sambungan yang peluangnya sekitar 9×10^{-8} per interval (Subbagian 2.3). Aliran rata-rata melintas dari blok kedua adalah $\pi_{129}T_{129,128} \approx 3.8 \times 10^{-3} \cdot 9.0 \times 10^{-8} \approx 3.4 \times 10^{-10}$ per interval, sedangkan blok itu menampung kira-kira separuh massa π . Waktu keluar rata-rata dari blok kedua karena itu berorde 10^9 interval, sesuai dengan $\kappa_1(B) \approx 1.7 \times 10^9$ karena $\|B\|_1$ berorde satu. Perhitungan ini hanya taksiran orde besar, tetapi menunjukkan bahwa lonjakan κ_1 pada $N = 256$ berasal dari struktur data, bukan dari algoritma.

7.3 Hasil

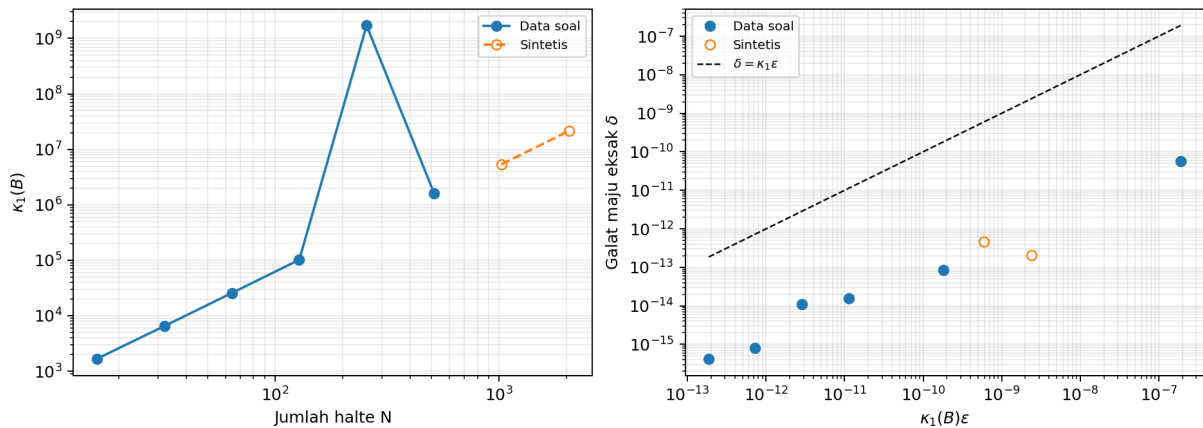
Seluruh nilai dihitung untuk kedua solver. Residual, η , e_{norm} , dan δ identik pada keduanya sampai semua angka yang disimpan (`hasil/kondisi_galat.csv`), sehingga tabel hanya memuat satu kolom untuk tiap besaran. Baris $N = 1024$ dan 2048 adalah rantai sintetis dari Bagian 6.

Tabel 7.1: Kondisi dan galat residual (κ_1 pada norma-1, r pada norma-2, η pada norma-1). Nilai sama untuk solver dense dan banded. Dua baris terakhir adalah rantai sintetis.

N	$\kappa_1(B)$	κ_1/N^2	r	$r/\ \hat{\pi}\ _2$	η	e_{norm}
16	1.68×10^3	6.55	1.55×10^{-17}	6.07×10^{-17}	1.31×10^{-17}	0
32	6.52×10^3	6.36	1.30×10^{-17}	7.21×10^{-17}	1.06×10^{-17}	0
64	2.57×10^4	6.27	9.66×10^{-18}	7.60×10^{-17}	1.09×10^{-17}	0
128	1.02×10^5	6.23	5.95×10^{-18}	6.62×10^{-17}	1.29×10^{-17}	0
256	1.71×10^9	2.61×10^4	3.85×10^{-18}	6.07×10^{-17}	9.88×10^{-18}	0
512	1.62×10^6	6.19	3.21×10^{-18}	7.15×10^{-17}	1.11×10^{-17}	0
1024	5.36×10^6	5.11	4.19×10^{-17}	1.34×10^{-15}	1.02×10^{-17}	0
2048	2.15×10^7	5.12	4.17×10^{-17}	1.89×10^{-15}	1.12×10^{-17}	0

Tabel 7.2: Galat maju terhadap solusi acuan eksak (δ pada norma-1) dan pembandingnya. Nilai δ sama untuk solver dense dan banded, dan $\max_i |\hat{\pi}_i^{\text{dense}} - \hat{\pi}_i^{\text{band}}| = 0$ pada semua N . $\varepsilon = 2^{-53}$.

N	$\kappa_1 \eta$	$\kappa_1 \varepsilon$	δ	$\delta/(\kappa_1 \varepsilon)$
16	2.20×10^{-14}	1.86×10^{-13}	4.24×10^{-16}	2.28×10^{-3}
32	6.92×10^{-14}	7.24×10^{-13}	8.02×10^{-16}	1.11×10^{-3}
64	2.80×10^{-13}	2.85×10^{-12}	1.09×10^{-14}	3.81×10^{-3}
128	1.32×10^{-12}	1.13×10^{-11}	1.57×10^{-14}	1.39×10^{-3}
256	1.69×10^{-8}	1.90×10^{-7}	5.75×10^{-11}	3.03×10^{-4}
512	1.79×10^{-11}	1.80×10^{-10}	8.64×10^{-14}	4.79×10^{-4}
1024	5.49×10^{-11}	5.95×10^{-10}	4.65×10^{-13}	7.82×10^{-4}
2048	2.40×10^{-10}	2.38×10^{-9}	2.03×10^{-13}	8.50×10^{-5}



Gambar 7.1: Kiri: $\kappa_1(B)$ terhadap N (skala log-log); garis putus-putus dan marker kosong adalah rantai sintetis. Kanan: galat maju eksak δ terhadap indikator $\kappa_1 \varepsilon$; semua titik berada di bawah garis $\delta = \kappa_1 \varepsilon$.

7.4 Analisis

Tabel 7.1 dan 7.2 menunjukkan empat pola.

Pertama, residual tidak mengikuti κ_1 . Residual r berada pada orde 10^{-18} – 10^{-17} dan η pada orde 10^{-17} untuk semua N , padahal $\kappa_1(B)$ merentang dari 1.68×10^3 sampai 1.71×10^9 . Pada data, $r/\|\hat{\pi}\|_2$ berada di 6.1 – 7.6×10^{-17} , kira-kira 0.5 – 0.7ε , jadi residual sudah sekecil yang dapat dicapai presisi ganda. Hal ini sesuai dengan sifat algoritma: B dominan diagonal per kolom, sehingga eliminasi Gauss tidak memerlukan pertukaran baris dan faktor pertumbuhan elemennya paling besar 2 [7], jadi stabil mundur. Pada rantai sintetis $r/\|\hat{\pi}\|_2$ sekitar 10^{-15} , lebih besar tetapi tetap kecil; penyebabnya tidak kami telusuri.

Kedua, galat maju mengikuti κ_1 sebagai batas atas. Nilai δ naik dari 4.24×10^{-16} ($N = 16$, $\kappa_1 = 1.68 \times 10^3$) menjadi 5.75×10^{-11} ($N = 256$, $\kappa_1 = 1.71 \times 10^9$). Pada seluruh baris $\delta < \kappa_1 \eta$ dan $\delta/(\kappa_1 \varepsilon)$ berada di antara 8.5×10^{-5} dan 3.8×10^{-3} , sehingga (7.3) berlaku tetapi konservatif sekitar dua sampai empat orde besaran. Hubungannya batas atas, bukan prediksi: δ pada $N = 2048$ (2.03×10^{-13}) lebih kecil daripada pada $N = 1024$ (4.65×10^{-13}) walaupun κ_1 -nya lebih besar. Residual yang kecil juga tidak menjamin galat maju yang kecil. Pada $N = 256$, $r = 3.85 \times 10^{-18}$ lebih kecil daripada pada $N = 16$ (1.55×10^{-17}), tetapi δ sekitar 1.4×10^5 kali lebih besar. Sebaliknya $e_{\text{norm}} = 0$ pada semua N dan kedua solver. Ini bukan bukti akurasi: normalisasi (3.3) memaksa $\mathbf{1}^T \hat{\pi} = 1$, jadi e_{norm} hanya memeriksa langkah normalisasi dan tidak menilai ketepatan komponen $\hat{\pi}_i$.

Ketiga, $N = 256$ adalah kasus kondisi buruk. Dari $N = 128$ ke 256 , κ_1 naik 1.7×10^4 kali dan δ naik 3.7×10^3 kali. Selisih terhadap `np.linalg.solve` pada $N = 256$ (6.6×10^{-13} , Bagian 5) juga jauh lebih besar daripada ukuran lain ($\leq 2.6 \times 10^{-16}$). Selisih itu sama ordenya dengan galat per komponen, $\delta/N \approx 2.2 \times 10^{-13}$. Dua eliminasi yang sama-sama benar tetapi berbeda urutan pembulatan berselisih sebesar yang diizinkan kondisi masalah. Kondisi buruk tidak berarti hasilnya cacat: $\hat{\pi}$ tetap mulus di sambungan ($\pi_{129}/\pi_{128} = 0.9986$) dan masih benar sampai sekitar sepuluh digit. Artinya $\hat{\pi}$ sensitif terhadap gangguan pada T dan pembulatan [8]. Berdasarkan Proposisi, κ_1 kira-kira berbanding terbalik dengan peluang sambungan; bila sambungan makin lemah dan $\kappa_1 \varepsilon$ mendekati satu, tidak ada digit yang dijamin benar.

Keempat, solver dense dan banded tidak dapat dibedakan. Selisih $\hat{\pi}$ keduanya nol pada semua N , dan r , η , e_{norm} , serta δ juga sama persis. Penyebabnya tidak ada pertukaran baris pada data (Bagian 5), sehingga kedua solver mengeksekusi pembagian dan pengurangan yang sama pada elemen tak-nol yang sama; solver dense hanya menambah operasi dengan nol, yang tidak mengubah hasil. Kesamaan persis ini berlaku pada mesin kami. Pada mesin atau pustaka lain, urutan penjumlahan pada substitusi dapat berbeda dan selisih berorde ε mungkin muncul, jauh di bawah δ . Kesimpulannya, galat solusi ditentukan oleh kondisi masalah dan algoritma eliminasi yang sama-sama dipakai kedua solver, bukan oleh cara penyimpanan matriks. Solver banded menghemat waktu dan memori tanpa mengorbankan akurasi.

8 Interpretasi Hasil

8.1 Prosedur

Distribusi yang dianalisis adalah $\hat{\pi}$ dari solver banded, yang identik dengan hasil solver dense (Bagian 7). Pada semua N , $\hat{\pi}_i > 0$ untuk setiap i dan $\mathbf{1}^T \hat{\pi} = 1$ (diperiksa dengan `assert` di notebook), sesuai teori untuk rantai irreducible [1]. Karena jumlah halte berbeda-beda, proporsi dibandingkan dengan distribusi seragam $1/N$ lewat $N\hat{\pi}_i$; nilai 1 berarti halte menampung penumpang sebanyak rata-rata. Kode 8.1 merangkum

perhitungan yang dipakai: halte dengan proporsi terbesar, halte yang berada dalam 1% dari maksimum, waktu kembali rata-rata $m_i = 1/\pi_i$ [1], drift per halte $\mu_i = \sum_j (j - i)T_{ij}$, pemeriksaan keseimbangan arus, dan ekstrapolasi Richardson. Ekstrapolasi dipakai karena nilai $N\hat{\pi}_{i^*}$ konvergen dengan galat $O(1/N)$ (Subbagian 8.2), sehingga $v_\infty \approx 2v(2N) - v(N)$ menghilangkan suku galat utamanya.

Kode 8.1 Ringkasan dan diagnosis distribusi steady state

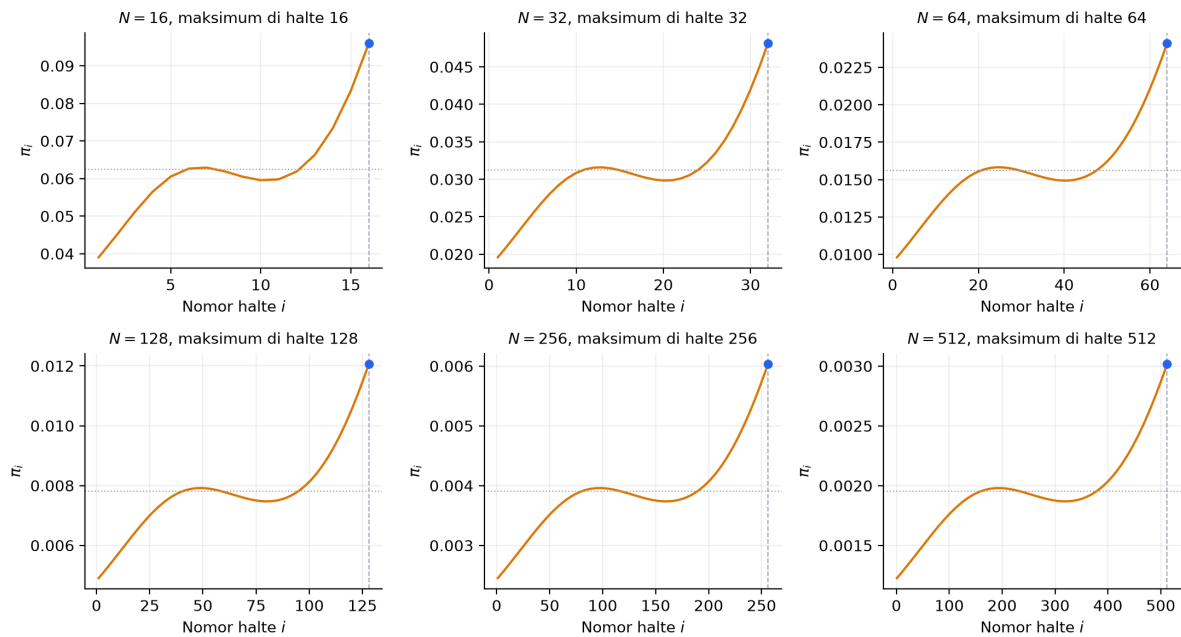
Masukan: T dan $\hat{\pi}$ untuk setiap $N \in \{16, 32, \dots, 512\}$

Keluaran: i^* , himpunan $\mathcal{H}$, m_{i^*} , drift μ , galat keseimbangan, v_∞

- 1: $i^* \leftarrow \arg \max_i \hat{\pi}_i$; $i_* \leftarrow \arg \min_i \hat{\pi}_i$
 - 2: $\mathcal{H} \leftarrow \{i : \hat{\pi}_i \geq 0.99 \hat{\pi}_{i^*}\}$ ▷ halte dalam 1% dari maksimum
 - 3: $m_{i^*} \leftarrow 1/\hat{\pi}_{i^*}$ ▷ waktu kembali rata-rata ke halte terpadat
 - 4: $\mu_i \leftarrow \sum_j (j - i)T_{ij}$ untuk semua i
 - 5: $g \leftarrow \max_i |\hat{\pi}_i(1 - T_{ii}) - \sum_{j \neq i} \hat{\pi}_j T_{ji}|$ ▷ keseimbangan arus; harus mendekati 0
 - 6: $v(N) \leftarrow N\hat{\pi}_{i^*}$; $v_\infty \leftarrow 2v(2N) - v(N)$ ▷ ekstrapolasi Richardson
 - 7: **return** $i^*, \mathcal{H}, m_{i^*}, \mu, g, v_\infty$
-

8.2 Hasil

Gambar 8.1 memuat $\hat{\pi}_i$ terhadap nomor halte untuk keenam ukuran, dan Tabel 8.1 merangkum nilai ekstremnya.



Gambar 8.1: Distribusi steady state $\hat{\pi}_i$ terhadap nomor halte. Garis titik-titik horizontal adalah distribusi seragam $1/N$; titik biru dan garis putus-putus vertikal menandai halte dengan proporsi terbesar.

Tabel 8.1: Nilai ekstrem $\hat{\pi}$. Halte terbesar i^* adalah halte N dan halte terkecil adalah halte 1 untuk semua N . Kolom “dalam 1%” adalah halte dengan $\hat{\pi}_i \geq 0.99 \hat{\pi}_{i^*}$, dan $1/\pi_{i^*}$ adalah waktu kembali rata-rata ke halte terpadat (dalam interval).

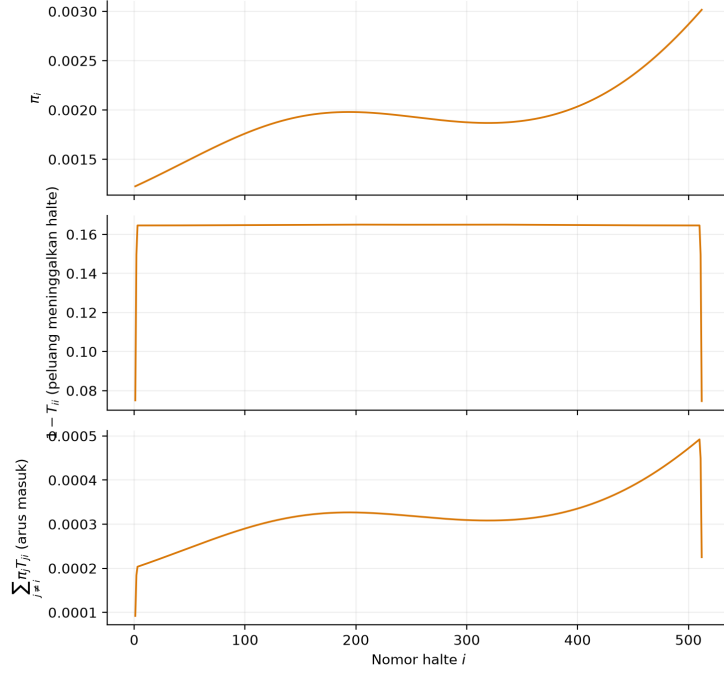
N	i^*	π_{i^*}	$N\pi_{i^*}$	π_1	$N\pi_1$	dalam 1%	$1/\pi_{i^*}$
16	16	0.095963	1.5354	0.039016	0.6243	16	10.42
32	32	0.048136	1.5403	0.019570	0.6263	32	20.77
64	64	0.024103	1.5426	0.009800	0.6272	64	41.49
128	128	0.012060	1.5437	0.004903	0.6276	128	82.92
256	256	0.006032	1.5442	0.002452	0.6278	255–256	165.78
512	512	0.003017	1.5445	0.001226	0.6279	510–512	331.50

Halte dengan proporsi terbesar adalah halte terakhir, yaitu halte N , pada keenam ukuran. Proporsinya turun dari 9.60% ($N = 16$) menjadi 0.302% ($N = 512$), tetapi $N\pi_N$ hampir konstan di sekitar 1.54: halte terakhir memuat sekitar 1.54 kali proporsi seragam. Halte terkecil adalah halte 1 dengan $N\pi_1 \approx 0.63$, atau sekitar 37% di bawah rata-rata, sehingga $\pi_N/\pi_1 \approx 2.46$ untuk semua N . Himpunan halte dalam 1% dari maksimum hanya satu halte untuk $N \leq 128$, dua untuk $N = 256$, dan tiga untuk $N = 512$, jadi hanya beberapa halte terakhir yang benar-benar terpadat. Sebagai gambaran (bukan data), bila 10 000 penumpang tersebar menurut π pada $N = 512$, halte 512 memuat sekitar 30 penumpang, halte 1 sekitar 12, dan halte rata-rata 19.5.

Bentuk $\hat{\pi}$ tidak monoton (Gambar 8.1). Proporsi naik dari halte 1 sampai maksimum lokal di sekitar $i/N \approx 0.4$ ($N\pi_i \approx 1.01$), turun ke minimum lokal di sekitar $i/N \approx 0.6$ (≈ 0.96), lalu naik tajam sampai halte terakhir. Nilai-nilai ini dibaca dari Gambar 8.3 (kanan), jadi hanya akurat sampai sekitar dua angka. Kurva $N\hat{\pi}_i$ terhadap i/N untuk keenam ukuran hampir berimpit pada gambar itu, sehingga bentuk distribusi hanya bergantung pada posisi relatif i/N . Konvergensinya dapat dilihat pada Tabel 8.1: selisih $N\pi_{i^*}$ antar ukuran berturut-turut adalah 4.9×10^{-3} , 2.3×10^{-3} , 1.1×10^{-3} , 5.3×10^{-4} , dan 2.6×10^{-4} , dengan rasio berurutan 2.17, 2.09, 2.04, dan 2.03, jadi galatnya $O(1/N)$. Ekstrapolasi Richardson memberi $N\pi_{i^*} \rightarrow 1.5448$ dan $N\pi_1 \rightarrow 0.6281$. Distribusi untuk koridor yang lebih panjang karena itu dapat ditaksir tanpa menyelesaikan sistemnya lagi: halte terakhir akan memuat sekitar $1.545/N$ dan halte pertama sekitar $0.628/N$. Taksiran ini hanya berlaku bila pola peluang transisi dipertahankan seperti pada data.

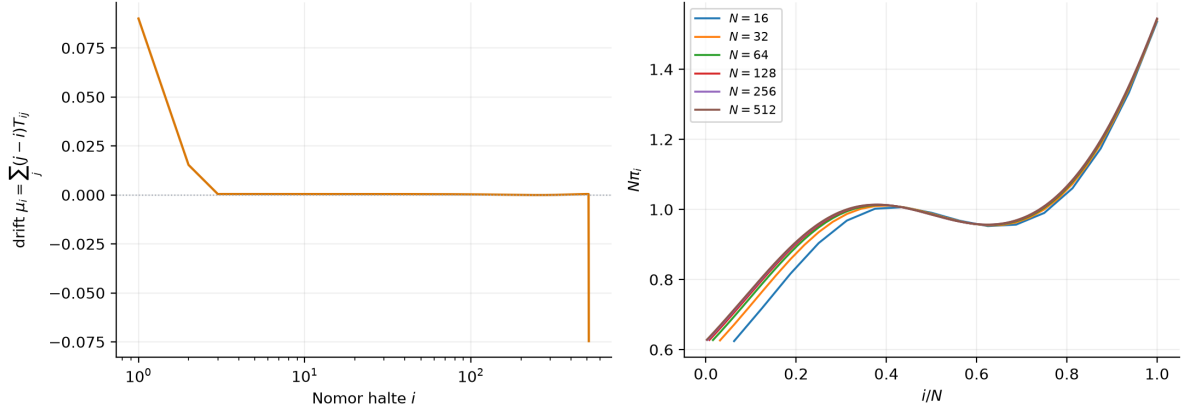
8.3 Mengapa halte terakhir terpadat

Gambar 8.2 (untuk $N = 512$) memeriksa dua penjelasan yang sederhana. Persamaan $T^T \pi = \pi$ pada halte i berarti arus keluar sama dengan arus masuk, $\pi_i(1 - T_{ii}) = \sum_{j \neq i} \pi_j T_{ji}$, dan selisih keduanya paling besar 1.6×10^{-18} pada $\hat{\pi}$ yang dihitung. Penjelasan pertama adalah lama tinggal: halte dengan peluang keluar $1 - T_{ii}$ kecil menahan penumpang lebih lama. Peluang keluar memang turun dari ≈ 0.165 di bagian dalam menjadi ≈ 0.075 di halte 1 dan halte 512, tetapi kedua halte itu sama-sama “lengket”, sementara π_1 terkecil dan π_{512} terbesar. Korelasi antara π_i dan $1/(1 - T_{ii})$ hanya 0.035. Jadi proporsi penumpang tidak ditentukan oleh lama tinggal, melainkan oleh aliran neto sepanjang koridor.



Gambar 8.2: Untuk $N = 512$: $\hat{\pi}_i$ (atas), peluang meninggalkan halte $1 - T_{ii}$ (tengah), dan arus masuk $\sum_{j \neq i} \hat{\pi}_j T_{ji}$ (bawah).

Penjelasan kedua adalah drift $\mu_i = \sum_j (j - i) T_{ij}$ (Gambar 8.3, kiri). Pada $N = 512$, μ_i adalah $+0.09$ di halte 1 dan -0.0746 di halte 512, jadi kedua ujung memantulkan penumpang ke dalam. Di bagian dalam koridor drift praktis nol: $\mu_i \approx 4 \times 10^{-4}$ pada halte 3–8 dan 505–510, dan $|\mu_i| \leq 2.8 \times 10^{-4}$ pada halte 100–412. Penumpang di sana berdifusi dengan variansi per interval $\sigma^2 \approx 0.21$ (Bagian 7), dan bentuk π ditentukan oleh drift kecil tersebut. Untuk difusi dengan drift μ dan variansi σ^2 , kepadatan stasioner memenuhi $d \ln \pi / di \approx 2\mu / \sigma^2$. Drift 4×10^{-4} menggeser $\ln \pi$ sekitar 0.4% per halte, yang menumpuk menjadi perubahan orde satu dalam beberapa ratus halte, sesuai dengan skala variasi π pada Gambar 8.2 (rasio $\pi_{512} / \pi_1 \approx 2.46$). Agar bentuk $N\hat{\pi}_i$ tidak bergantung pada N , drift bagian dalam harus mengecil sebanding $1/N$. Hal ini konsisten dengan $N\mu \approx 0.2$ pada $N = 512$, tetapi kami tidak memeriksa μ_i untuk N lain. Karena itu penjelasan ini kami nyatakan sebagai tafsiran yang konsisten dengan data, bukan sebagai bukti.



Gambar 8.3: Kiri: drift $\mu_i = \sum_j (j - i) T_{ij}$ untuk $N = 512$ (sumbu horizontal logaritmik). Kanan: $N\hat{\pi}_i$ terhadap i/N untuk keenam ukuran; kurva hampir berimpit.

8.4 Implikasi operasional

Bagi perusahaan, halte yang paling berisiko penuh adalah beberapa halte terakhir koridor: $N\pi_i \geq 1.52$ pada halte-halte dalam 1% dari maksimum, dan proporsinya terus naik menuju ujung. Halte terakhir memuat sekitar 1.54 kali rata-rata dan halte pertama hanya sekitar 0.63 kali, sehingga kapasitas halte, armada pengangkut ke arah ujung, dan petugas sebaiknya dialokasikan tidak merata. Waktu kembali rata-rata ke halte terpadat adalah 331.5 interval pada $N = 512$, dibandingkan sekitar 815 interval ke halte 1 ($1/\pi_1$), jadi penumpang kembali ke halte terakhir sekitar 2.5 kali lebih sering.

9 Rekomendasi

Rekomendasi didasarkan pada empat kriteria: waktu, memori, kompleksitas, dan galat. Tabel 9.1 merangkum bukti dari Bagian 6 dan 7.

Tabel 9.1: Perbandingan solver dense dan banded ($p = 2, q = 1$). Waktu dan memori eksperimen diambil dari Tabel 6.1; galat dari Tabel 7.1 dan 7.2.

Kriteria	Dense LU	Banded LU	Keunggulan
Waktu faktorisasi (teori)	$\approx N^3/3$ flop, $O(N^3)$	$\approx Np(p+q) = 6N$ flop, $O(N)$	banded
Memori (teori)	$8N^2$ byte	$8(2p+q+1)N = 48N$ byte	banded
Waktu solver, $N = 512$ (data)	8.2 kali waktu banded	20.1 ms	banded, 8.2 kali
Waktu solver, $N = 2048$ (sintetis)	12.7 s	83 ms	banded, sekitar 152 kali
Memori <code>.nbytes</code> , $N = 512$	4108 KiB	60 KiB	banded, sekitar 68 kali
Memori <code>.nbytes</code> , $N = 2048$	65584 KiB	240 KiB	banded, sekitar 270 kali
Galat maju δ , $N = 256$ (terburuk)	5.75×10^{-11}	5.75×10^{-11}	sama
Residual r , semua N	$\leq 4.2 \times 10^{-17}$	$\leq 4.2 \times 10^{-17}$	sama
Pengaruh $\kappa_1(B)$ pada galat	sama	sama	tidak membedakan

Kami merekomendasikan solver banded (eliminasi pita dengan *partial pivoting*, Kode 5.2) sebagai metode utama untuk koridor dengan jumlah halte besar. Alasannya ada tiga. Pertama, kompleksitasnya linear: waktu $O(Np(p+q))$ dan memori $O((2p+q+1)N)$ untuk p dan q tetap, sedangkan solver dense $O(N^3)$ dan $O(N^2)$. Bedanya terasa di ukuran yang realistis: untuk $N = 10^5$, matriks dense membutuhkan 8×10^{10} byte (sekitar 80 GB) dan sekitar 3.3×10^{14} flop untuk faktorisasi, sedangkan array pita hanya sekitar 4.8 MB dan 6×10^5 flop (ekstrapolasi dari rumus, bukan eksperimen). Kedua, akurasi sama: δ , r , dan η identik dengan solver dense pada semua N (Bagian 7), sehingga penghematan tidak dibayar dengan galat. Ketiga, solver ini umum: p dan q dideteksi otomatis, tidak harus tridiagonal, dan *partial pivoting* tetap dipertahankan sehingga tetap aman bila T lain tidak dominan diagonal; fill-in dibatasi pada $p+q$ diagonal di atas diagonal utama.

Ada empat catatan agar rekomendasi ini dipakai dengan benar. Pertama, titik silang di sekitar $N = 128$ pada Tabel 6.1 adalah artefak implementasi Python (perulangan kecil per langkah), bukan sifat algoritma. Jumlah operasi banded jauh lebih sedikit sejak $N = 16$ (Tabel 6.2 dan `hasil/flop_teoris.csv`). Pada kode terkompilasi, misalnya rutin pita LAPACK [6], solver banded diharapkan unggul pada N kecil juga, tetapi hal ini tidak kami uji. Kedua, setelah solver menjadi linear, pembacaan dan persiapan data menjadi hambatan: pada $N = 512$, membaca CSV dense (29.4 ms) sudah melebihi waktu solver banded (20.1 ms), dan deteksi p, q dari matriks dense berbiaya $O(N^2)$. Seluruh alur menjadi $O(N)$ bila T disimpan langsung sebagai empat diagonalnya (offset $-1, 0, 1, 2$).

Ketiga, kondisi harus dipantau. $\kappa_1(B)$ dapat ditaksir dalam $O(N)$ untuk matriks pita memakai estimator Hager–Higham dengan faktor yang sudah ada [7]. Kasus $N = 256$ menunjukkan bahwa rantai yang hampir terpecah membuat $\kappa_1 \approx 1.7 \times 10^9$ walaupun δ masih 5.75×10^{-11} . Bila $\kappa_1 \varepsilon$ melebihi toleransi galat yang dibutuhkan, ada dua pilihan: perbaikan iteratif yang memakai ulang faktor LU (setiap iterasi $O(N)$ pada solver banded, seperti pada Kode 7.2 tetapi dengan residual presisi ganda), atau algoritma Grassmann–

Taksar–Heyman (GTH), yang tidak memakai pengurangan dan dikenal akurat pada rantai hampir terpecah [9]. Kedua pilihan itu tidak diuji pada laporan ini. Keempat, iterasi pangkat $\pi \leftarrow T^T \pi$ tidak kami rekomendasikan: karena rantai berdifusi (Bagian 7), waktu campurnya umumnya berorde N^2 interval, sehingga biaya totalnya sekitar $O(N^3)$ meskipun tiap iterasi hanya $O(N)$ (juga tidak diuji). Solver dense tetap berguna sebagai pembandingan pada N kecil dan untuk matriks tanpa struktur pita.

Alur yang kami sarankan dirangkum pada Kode 9.1.

Kode 9.1 Alur yang disarankan untuk menghitung distribusi steady state

Masukan: T (atau empat diagonalnya), toleransi galat relatif τ

Keluaran: $\hat{\pi}$, ukuran galat r dan e_{norm} , penanda kondisi

```

1: validasi  $T$ : dimensi, nonnegatif, jumlah baris, irreducible (Bagian 2)
2: deteksi  $p, q$  dari  $B$  (Bagian 4)
3: if  $p + q + 1 \ll N$  then
4:   bentuk  $AB \in \mathbb{R}^{(2p+q+1) \times N}$  dan selesaikan  $Bz = b$  dengan Kode 5.2
5: else
6:   bentuk  $B$  dense dan selesaikan dengan Kode 5.1
7: end if
8:  $\hat{\pi} \leftarrow z / \mathbf{1}^T z$ ; hitung  $r = \|T^T \hat{\pi} - \hat{\pi}\|_2$  dan  $e_{\text{norm}}$ 
9: taksir  $\kappa_1(B)$  ▷ eksak (Kode 7.1) bila  $N$  kecil; estimator bila  $N$  besar
10: if  $\kappa_1(B) \varepsilon > \tau$  then
11:   lakukan perbaikan iteratif atau gunakan algoritma GTH, dan tandai hasil sebagai
       berkondisi buruk
12: end if
13: return  $\hat{\pi}, r, e_{\text{norm}}, \kappa_1(B)$ 

```

10 Kesimpulan

Enam matriks transisi valid dan menghasilkan distribusi stasioner tunggal. Struktur B memiliki $p = 2$ dan $q = 1$, sehingga penyimpanan dan eliminasi pita memerlukan ruang dan operasi yang tumbuh linear terhadap N untuk lebar pita tetap. Pada data $N = 512$, solver banded memakai 60 KiB dan sekitar 20.1 ms, sedangkan solver dense memakai 4108 KiB dan sekitar 164.2 ms. Kedua solusi memberikan residual stasioner yang praktis identik; pada $N = 256$ kondisi sangat besar karena rantai hampir terpisah, sehingga pemeriksaan kondisi tetap penting. Untuk koridor yang panjang, solver banded dengan partial pivoting adalah pilihan utama.

Referensi

- [1] J. R. Norris, *Markov Chains*. Cambridge University Press, 1997, Bab 1.3 dan 1.7–1.8.
- [2] T. H. Cormen, C. E. Leiserson, R. L. Rivest, dan C. Stein, *Introduction to Algorithms*, edisi ke-3. MIT Press, 2009.
- [3] M. T. Heath, *Scientific Computing: An Introductory Survey*, edisi ke-2. McGraw-Hill, 2002, Bab 1–2.

- [4] W. J. Stewart, *Introduction to the Numerical Solution of Markov Chains*. Princeton University Press, 1994.
- [5] G. H. Golub dan C. F. Van Loan, *Matrix Computations*, edisi ke-4. Johns Hopkins University Press, 2013, Bab 3 dan Subbab 4.3.
- [6] E. Anderson dkk., *LAPACK Users' Guide*, edisi ke-3. SIAM, 1999.
- [7] N. J. Higham, *Accuracy and Stability of Numerical Algorithms*, edisi ke-2. SIAM, 2002, Bab 7, 9, 12, dan 15.
- [8] G. E. Cho dan C. D. Meyer, "Comparison of perturbation bounds for the stationary distribution of a Markov chain," *Linear Algebra and its Applications*, vol. 335, hlm. 137–150, 2001.
- [9] W. K. Grassmann, M. I. Taksar, dan D. P. Heyman, "Regenerative analysis and steady state distributions for Markov chains," *Operations Research*, vol. 33, no. 5, hlm. 1107–1116, 1985.

A Kode Program

Program lengkap berada di `soal1/soal1.ipynb` dan dijalankan dengan *Restart & Run All*; seluruh tabel dan grafik disimpan ke `soal1/hasil/`. Berikut fungsi inti validasi, kedua solver, dan pengukuran waktu, disalin apa adanya dari notebook.

```
def bfs_jumlah_dikunjungi(T: np.ndarray, balik: bool = False) -> int:
    N = T.shape[0]
    dikunjungi = np.zeros(N, dtype=bool)
    dikunjungi[0] = True
    antrean = deque([0])
    while antrean:
        u = antrean.popleft()
        # graf asli: tetangga dari baris u; graf dibalik: dari kolom u
        if balik:
            tetangga = np.nonzero(T[:, u] > 0)[0]
        else:
            tetangga = np.nonzero(T[u, :] > 0)[0]
        for v in tetangga:
            if not dikunjungi[v]:
                dikunjungi[v] = True
                antrean.append(v)
    return int(dikunjungi.sum())

def validasi_T(T: np.ndarray, N: int, tol: float = TOL_BARIS) -> dict:
    hasil = {'N': N}
    hasil['dimensi_ok'] = T.shape == (N, N)
    hasil['hingga'] = bool(np.all(np.isfinite(T)))
    if not (hasil['dimensi_ok'] and hasil['hingga']):
        hasil['valid'] = False
        return hasil

    hasil['nonnegatif'] = bool(T.min() >= 0)
    hasil['galat_baris_maks'] = float(np.max(np.abs(T.sum(axis=1) - 1)))
    hasil['baris_ok'] = hasil['galat_baris_maks'] <= tol
    # terhubung kuat, semua halte tercapai dari halte 1 DAN halte 1 tercapai dari semua halte
    hasil['irreducible'] = (bfs_jumlah_dikunjungi(T) == N
                           and bfs_jumlah_dikunjungi(T, balik=True) == N)
    hasil['aperiodik'] = bool(np.any(np.diag(T) > 0))
    hasil['min_diag'] = float(np.diag(T).min())
    # peluang pindah terkecil yang masih positif; sangat kecil = rantai "hampir terputus"
    luar_diag = T - np.diag(np.diag(T))
    hasil['min_takno_luar_diag'] = float(luar_diag[luar_diag > 0].min()) if np.any(luar_diag > 0) else 0.0
```

```

    nnz = np.count_nonzero(T, axis=1)
    hasil['nnz_per_baris'] = f'{nnz.min()}--{nnz.max()}'
    i, j = np.nonzero(T)
    hasil['offset_taknol'] = str(sorted(set((j - i).tolist()))) # nilai j - i dari elemen tak-nol

    hasil['valid'] = hasil['nonnegatif'] and hasil['baris_ok'] and hasil['irreducible']
    return hasil

def bentuk_B_dense(T: np.ndarray) -> tuple[np.ndarray, np.ndarray]:
    N = T.shape[0]
    B = np.eye(N) - T.T          #  $A = I - T^T$ 
    B[0, :] = 0.0                # baris 1 diganti [1 0 ... 0]
    B[0, 0] = 1.0
    b = np.zeros(N)
    b[0] = 1.0
    return B, b

def deteksi_bandwidth(M: np.ndarray) -> tuple[int, int]:
    '''Menentukan bandwidth dari posisi elemen tak-nol.

    Input : M (N x N).
    Output: p = maks(i - j) (lower), q = maks(j - i) (upper).
    '''
    i, j = np.nonzero(M)
    p = max(0, int(np.max(i - j)))
    q = max(0, int(np.max(j - i)))
    return p, q

def lu_dense(B: np.ndarray) -> tuple[np.ndarray, np.ndarray]:
    LU = B.astype(float).copy()
    N = LU.shape[0]
    perm = np.arange(N)
    for k in range(N - 1):
        # pivot: elemen dengan nilai mutlak terbesar di kolom k, baris k..N-1
        r = k + int(np.argmax(np.abs(LU[k:, k])))
        if LU[r, k] == 0.0:
            raise ValueError(f'B singular: kolom {k} tidak punya pivot tak-nol')
        if r != k:
            LU[[k, r], :] = LU[[r, k], :] # tukar seluruh baris, termasuk multiplier lama
            perm[[k, r]] = perm[[r, k]]
        # multiplier  $L_{ik}$  disimpan di tempat elemen yang dieliminasi
        LU[k + 1:, k] /= LU[k, k]
        # baris i dikurangi  $L_{ik}$  * baris k (kolom k+1 ke kanan)
        LU[k + 1:, k + 1:] -= np.outer(LU[k + 1:, k], LU[k, k + 1:])
    if LU[N - 1, N - 1] == 0.0:
        raise ValueError('B singular: pivot terakhir nol')
    return LU, perm

def solve_lu(LU: np.ndarray, perm: np.ndarray, b: np.ndarray) -> np.ndarray:
    N = LU.shape[0]
    y = b[perm].astype(float) #  $Pb$ 
    for i in range(1, N): # substitusi maju  $Ly = Pb$  (diagonal  $L = 1$ )
        y[i] -= np.dot(LU[i, :i], y[:i])
    z = np.zeros(N)
    for i in range(N - 1, -1, -1): # substitusi mundur  $Uz = y$ 
        z[i] = (y[i] - np.dot(LU[i, i + 1:], z[i + 1:])) / LU[i, i]
    return z

def deteksi_bandwidth_T(T: np.ndarray) -> tuple[int, int]:
    i, j = np.nonzero(T)
    pakai = j >= 1
    p = max(0, int(np.max(j[pakai] - i[pakai])))
    q = max(0, int(np.max(i[pakai] - j[pakai])))
    return p, q

def bentuk_B_band(T: np.ndarray, p: int, q: int) -> tuple[np.ndarray, np.ndarray]:
    N = T.shape[0]
    d = p + q # baris AB tempat diagonal utama
    AB = np.zeros((2 * p + q + 1, N))
    AB[d, 0] = 1.0 # baris 1 B = [1 0 ... 0]
    for i in range(1, N):
        for j in range(max(0, i - p), min(N - 1, i + q) + 1):

```

```

        AB[d + i - j, j] = (1.0 if i == j else 0.0) - T[j, i]
    b = np.zeros(N)
    b[0] = 1.0
    return AB, b

def lu_band(AB: np.ndarray, p: int, q: int) -> tuple[np.ndarray, np.ndarray]:
    AB = AB.copy()
    d = p + q
    N = AB.shape[1]
    piv = np.zeros(N, dtype=int)
    for k in range(N):
        i_maks = min(N - 1, k + p)
        j_maks = min(N - 1, k + p + q)
        r = k + int(np.argmax(np.abs(AB[d:d + i_maks - k + 1, k])))
        if AB[d + r - k, k] == 0.0:
            raise ValueError(f'B singular: kolom {k} tidak punya pivot tak-nol')
        piv[k] = r
        js = np.arange(k, j_maks + 1)
        if r != k:
            sementara = AB[d + k - js, js]
            AB[d + k - js, js] = AB[d + r - js, js]
            AB[d + r - js, js] = sementara
        js = js[1:]
        for i in range(k + 1, i_maks + 1):
            AB[d + i - k, k] /= AB[d, k]
            AB[d + i - js, js] -= AB[d + i - k, k] * AB[d + k - js, js]
    return AB, piv

def solve_band(AB: np.ndarray, piv: np.ndarray, p: int, q: int, b: np.ndarray) -> np.ndarray:
    d = p + q
    N = AB.shape[1]
    y = b.astype(float).copy()
    for k in range(N):
        r = piv[k]
        if r != k:
            y[k], y[r] = y[r], y[k]
        for i in range(k + 1, min(N - 1, k + p) + 1):
            y[i] -= AB[d + i - k, k] * y[k]
    z = np.zeros(N)
    for i in range(N - 1, -1, -1):
        js = np.arange(i + 1, min(N - 1, i + p + q) + 1)
        z[i] = (y[i] - np.dot(AB[d + i - js, js], z[js])) / AB[d, i]
    return z

def ukur_waktu(fungsi, ulang: int = ULANG) -> float:
    fungsi()
    waktu = []
    for _ in range(ulang):
        t0 = time.perf_counter()
        fungsi()
        waktu.append(time.perf_counter() - t0)
    return float(np.min(waktu))

def ukur_peak(fungsi):
    tracemalloc.start()
    hasil = fungsi()
    _, peak = tracemalloc.get_traced_memory()
    tracemalloc.stop()
    return peak, hasil

```